Book Rating Prediction

Introduction

Online reading platforms contain massive collections of books along with ratings provided by readers. These ratings help readers decide which books to read and help publishers understand the popularity of their books. Predicting ratings using machine learning techniques can assist recommendation systems and publishing analytics.

This project aims to build a machine learning system that predicts book ratings using features extracted from a dataset. The trained model is then deployed as a web service that can accept book information and return predicted ratings. The system also integrates experiment tracking and model management using MLflow to ensure reproducibility.

The project also demonstrates the importance of model deployment. Many machine learning projects stop after model training, but real-world applications require models to be deployed so they can be used by other software systems.

Machine learning has become an important tool in modern data-driven applications. By analyzing historical data, machine learning models can identify relationships between different variables and use them to make predictions on unseen data. In the context of book platforms, features such as the number of ratings received by a book and the average rating given by readers can provide valuable insights into the popularity and reception of a book. Using these features, it is possible to develop a predictive model that estimates the likely rating of a book.

Objectives

The main objectives of this project are:

1. To analyze book rating data and identify useful features for prediction.
2. To train a machine learning model capable of predicting book ratings.
3. To track experiments and model parameters using MLflow.
4. To serialize the trained model for later usage.
5. To deploy the trained model using FastAPI as a REST API.
6. To containerize the application using Docker.
7. To create a reproducible environment using requirements and environment configuration files.

Technology Stack

1. Python is used as the main programming language for implementing the machine learning pipeline and backend API.
2. Pandas and NumPy are used for data processing and numerical operations. These libraries are widely used in data science workflows

3. Scikit-learn is used for building the machine learning model. It provides simple and efficient tools for predictive data analysis.
4. MLflow is used for experiment tracking and model management. It helps store model artifacts, track parameters, and visualize experiments.
5. FastAPI is used for building the web API that exposes the machine learning model for predictions.
6. Uvicorn is used as the ASGI server to run the FastAPI application.
7. Docker is used to package the entire application along with its dependencies so that it can run consistently across different environments.
8. Git and GitHub are used for version control and collaboration.

Dataset Description

The dataset used in this project contains information about books and their ratings. Book datasets usually include features such as title, author, ratings count, average rating, and the score count of the book

In this project, selected numerical features such as RatingsCount and AverageRating are used for training the model. These features provide useful information about how a book is perceived by readers and how popular it is.

Data preprocessing includes cleaning the dataset, removing missing values, and selecting the required features for model training.

And the data scraped from the website Goodreads, which contains the details about the books and here we scraped the particular type of books from the website. we scraped the feminist type of book and save it as Data frame and CSV file.

Web Scraping using requests and Beautiful Soup

Installing Required Libraries

!pip install --upgrade pip

!pip install --upgrade numpy

!pip install --upgrade pandas

!pip install seaborn

!pip install requests

!python -m pip install beautifulsoup4

Scraping the data

```
#Importing required libraries
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import re
BASE_URL = "https://www.goodreads.com/list/show/62.Best_Feminist_Books?page="
headers = {
    "User-Agent": "Mozilla/5.0"
}
books = []
for page in range(1, 7):
    url = BASE_URL + str(page)
    resp = requests.get(url, headers=headers)
    soup = BeautifulSoup(resp.text, "html.parser")
    rows = soup.find_all("tr", {"itemtype": "http://schema.org/Book"})
    for r in rows:
        title = r.find("a", class_="bookTitle").get_text(strip=True)
        author = r.find("a", class_="authorName").get_text(strip=True)
        rating_text = r.find("span", class_="minirating").get_text(strip=True)
        avg_rating = re.search(r"(\d\.\d+)", rating_text)
        avg_rating = avg_rating.group(1) if avg_rating else None
        ratings_count = re.search(r"([\d,]+)\sratings", rating_text)
        ratings_count = ratings_count.group(1) if ratings_count else None
        score_block = r.find("span", class_="smallText uitext")
```

```
        score = None
        if score_block:
            text = score_block.get_text(" ", strip=True)
            score_match = re.search(r"score:\s*([\d,]+)", text)
            votes_match = re.search(r"(\d+)\svotes", text)
            score = score_match.group(1) if score_match else None
        books.append({
            "Title": title,
            "Author": author,
            "AverageRating": avg_rating,
            "RatingsCount": ratings_count,
            "Score": score,
        })
    time.sleep(1)
#Converting the scraped data Dataframe
df = pd.DataFrame(books)
#saving the dataframe into csv file
df.to_csv("feminist_books.csv", index=False)
print("Saved successfully!")
```

Data Exploration

df.head() → printing the first few rows of the dataset
df.shape →printing the shape of the dataset
df.info() →printing the information about the dataset

Data Cleaning

Changing the Data type of the 3 features

```
df["AverageRating"] = pd.to_numeric(df["AverageRating"], errors="coerce")
df['RatingsCount'] = (
    df['RatingsCount']
    .astype(str)
    .str.replace(',', '', regex=False)
)

df['RatingsCount'] = pd.to_numeric(df['RatingsCount'], errors='coerce')
df['RatingsCount'] = df['RatingsCount'].fillna(0).astype(int)

df['Score'] = (
    df['Score']
    .astype(str)
    .str.replace(',', '')
    .str.strip()
)

df['Score'] = pd.to_numeric(df['Score'], errors='coerce')
df.info()
```

Duplicate Checking

#checking the unique values

```
df['Title'].nunique()
#printing the duplicate values from the title column
 df[df.duplicated('Title')]
#dropping the duplicate rows
df = df.drop_duplicates(subset='Title')
```

Statistical Summary

#describe the data

```
df.describe(include="all")
```

Missing Values

#checking if there is any missing values

```
df.isnull().sum()
```

Exploratory Data Analysis

Data Visualization

#histogram → it describes how the data is distributed

```
df.hist(figsize=(10,10))
```

#BoxPlot → it used to predict the outliers

```
df.boxplot(column=["RatingsCount"], figsize=(5,5))
```

 # Scatter plot → to represent the values of two different numeric variables

```
.plt.scatter(df["RatingsCount"], df["AverageRating"])
plt.xlabel("RatingsCount")
plt.ylabel("AverageRating")
plt.show()
```

Correlation Heatmap

#checking the correlation between the multiple variables

```
corr = df[["AverageRating", "RatingsCount", "Score"]].corr()
plt.figure(figsize=(6,4))
sns.heatmap(corr, annot=True, cmap="viridis", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Hypothesis Test

Pearson Hypothesis test:

The Pearson correlation hypothesis test determines if a linear relationship exists between two continuous, normally distributed variables by testing if the population correlation coefficient () is significantly different from zero.

```
#importing the required libraries
from scipy.stats import pearsonr
x = df["RatingsCount"]
y = df["Score"]
correlation_coefficient, p_value = pearsonr(x, y)
print ("Correlation:", correlation_coefficient)
print ("P-value:", p_value)
```

Explanation:

Pearson correlation is used to measure the strength and direction of the linear relationship between two variables. In this project, it is used to analyze the relationship between the number of ratings a book receives and its overall score. The correlation coefficient indicates the strength of the relationship, while the p-value determines whether the relationship is statistically significant.

Machine Learning Workflow

The machine learning workflow consists of several steps:

1. Data Loading – The dataset is loaded using Pandas.
2. Data Cleaning – Missing values and inconsistent records are removed.
3. Feature Selection – Important features are selected for model training.
4. Model Training – A regression model from Scikit-learn is trained.
5. Model Evaluation – The model is evaluated using appropriate metrics.
6. Model Serialization – The trained model is saved as a pickle file.
7. Experiment Tracking – MLflow logs parameters, metrics, and artifacts.

Machine Learning workflow

```
#Importing all the libraries

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
#selecting target variable and feature variable
X = df[["RatingsCount", "AverageRating"]]
y = df["Score"]
#spliting the dataset for testing and traning
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Linear Regression

→Linear Regression is a **supervised machine learning algorithm** used to predict a **continuous numerical value** based on one or more input variables.

→It works by identifying the **linear relationship between independent variables (features) and a dependent variable (target)**.

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("R2:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

Random Forest

→Random Forest Regressor is a **supervised machine learning algorithm** used for **regression problems**, where the goal is to predict a **continuous numerical value**. It is part of the Random Forest family and is built using multiple **decision trees**

```
#Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("Random Forest R2:", r2_score(y_test, y_pred_rf))
```

Decision Tree

→Decision Tree Regression is a **supervised machine learning algorithm** used to predict **continuous numerical values**. It works by splitting the dataset into smaller groups based on feature values and creating a **tree-like structure** that represents decision rules

```
from sklearn.tree import DecisionTreeRegressor
dt = DecisionTreeRegressor(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("Decision Tree R2:", r2_score(y_test, y_pred_dt))
```

Ridge Regression

→Ridge Regression is a **regularized version of Linear Regression** used to improve model performance when there is **multicollinearity** (strong correlation between input features) or when the model tends to **overfit the training data**.

#Ridge regression →It is used to predict book scores using RatingsCount and AverageRating.

```
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
print("Ridge R2:", r2_score(y_test, y_pred_ridge))
```

Lasso Regression

#Lasso Regression →Lasso Regression stands for **Least Absolute Shrinkage and Selection Operator**. It is a type of **linear regression that uses regularization** to improve model performance and prevent overfitting.

#Code

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
print("Lasso R2:", r2_score(y_test, y_pred_lasso))
```

Model Comparison

- ➔ Comparing all the trained models based on the r2 score values

```
#importing all the libraries
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.01),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42)
}
results = []
#loop for comparing the results
for name, model in models.items():
     model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    results.append({
        "Model": name,
```

```
        "R2 Score": r2,
        "MSE": mse
    })
    print(f"{name} --> R2: {r2:.4f}, MSE: {mse:.2f}")
```

MAE Calculation :

→MAE Calculation stands for Mean Absolute Error.

→**Mean Absolute Error (MAE)** is a metric used to measure how accurate a regression model is. It calculates the **average of the absolute differences between the actual values and the predicted values.**

#Code

```
from sklearn.metrics import mean_absolute_error
lin_reg = model
models = {
    "Linear Regression": lin_reg,
    "Ridge": ridge,
    "Lasso": lasso,
    "Decision Tree": dt,
    "Random Forest": rf
}
for name, model in models.items():
      y_pred = model.predict(X_test)
      mae = mean_absolute_error(y_test, y_pred)
      print(f"{name} --> MAE: {round(mae, 2)}")
```

Model Comparison:

→In this project different regression models were used to predict the **Score of books** using features like **RatingsCount and AverageRating**. The models used include:

- Linear Regression
- Ridge Regression
- Lasso Regression
- Decision Tree Regressor
- Random Forest Regressor

Each model was trained using the training dataset and then evaluated using the test dataset.

#Code

```
models = ['Linear Regression', 'Ridge', 'Lasso', 'Decision Tree', 'Random Forest']
```

```
r2_scores = [
    r2_score(y_test, y_pred),
    r2_score(y_test, y_pred_ridge),
    r2_score(y_test, y_pred_lasso),
    r2_score(y_test, y_pred_dt),
    r2_score(y_test, y_pred_rf)
]
mse_scores = [
    mean_squared_error(y_test, y_pred),
    mean_squared_error(y_test, y_pred_ridge),
    mean_squared_error(y_test, y_pred_lasso),
    mean_squared_error(y_test, y_pred_dt),
    mean_squared_error(y_test, y_pred_rf)
]
mae_scores = [
    mean_absolute_error(y_test, y_pred),
    mean_absolute_error(y_test, y_pred_ridge),
    mean_absolute_error(y_test, y_pred_lasso),
    mean_absolute_error(y_test, y_pred_dt),
    mean_absolute_error(y_test, y_pred_rf)
]
model_comparison = pd.DataFrame({
    'Model': models,
    'R2 Score': r2_scores,
    'MSE': mse_scores,
    'MAE': mae_scores
})
best_model = model_comparison.loc[model_comparison['R2 Score'].idxmax()]
print("Model Comparison Table:")
print(model_comparison)
print("\nBest Model based on R2 Score:")
print(best_model)
```

It prints the best model as linear regression.

Hyperparameter Tuning:

→**Optuna** is a Python library used for **hyperparameter optimization** in machine learning models. It helps find the **best combination of model parameters automatically** to improve model performance.

→The best modelis linear regression and it doesn't have the parameters to tune so we choose the another model with high R2 score value which is the Random Forest Regressor.

```
#Importing all Libraries
import optuna
from sklearn.model_selection import cross_val_score
```

```
import numpy as np
def objective_rf(trial):
    n_estimators = trial.suggest_int('n_estimators', 50, 500)
    max_depth = trial.suggest_int('max_depth', 3, 20)
    min_samples_split = trial.suggest_int('min_samples_split', 2, 20)
    min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 10)
    model = RandomForestRegressor(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf,
        random_state=42,
        n_jobs=-1
    )
   score = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring='r2'
    )
   return np.mean(score)
study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=50)
print ("Best Parameters for Random Forest:", study_rf.best_params)
print ("Best R2 Score for Random Forest:", study_rf.best_value)
```

Train Tuned Model: (Random forest)

```
best_rf_params = study_rf.best_params
best_rf_model = RandomForestRegressor(
    **best_rf_params,
    random_state=42,
    n_jobs=-1
)
best_rf_model.fit(X_train, y_train)
y_pred_best_rf = best_rf_model.predict(X_test)
r2_best_rf = r2_score(y_test, y_pred_best_rf)
mse_best_rf = mean_squared_error(y_test, y_pred_best_rf)
mae_best_rf = mean_absolute_error(y_test, y_pred_best_rf)
print("--- Tuned Random Forest Model Evaluation ---")
print(f"R2 Score: {r2_best_rf:.4f}")
print(f"Mean Squared Error: {mse_best_rf:.2f}")
print(f"Mean Absolute Error: {mae_best_rf:.2f}")
```

MLflow Integration

MLflow plays a critical role in this project by tracking machine learning experiments. During model training, MLflow records parameters such as hyperparameters used during training. It also logs evaluation metrics that describe the performance of the model. The trained model itself is saved as an artifact within MLflow.

MLflow also provides a user interface where experiments can be visualized and compared. This allows developers to identify which model version performs better and helps maintain a proper model lifecycle.

MLFLOW Logging:

```
import mlflow
import mlflow.sklearn
# Start MLflow run
with mlflow.start_run()
# Log model parameters
mlflow.log_param("model", "RandomForestRegressor")
mlflow.log_param("n_estimators", rf.n_estimators)
mlflow.log_param("max_depth", rf.max_depth)
# Log metrics
mlflow.log_metric("R2_score", r2_best_rf)
mlflow.log_metric("MSE", mse_best_rf)
mlflow.log_metric("MAE", mae_best_rf)
# Log the trained model
mlflow.sklearn.log_model(rf, "random_forest_model")
print ("MLflow logging completed")
```

Saving the Logged model

```
import mlflow
import mlflow.sklearn
```

```
mlflow.sklearn.save_model(rf, "model")

print("Random Forest saved successfully!")

mlflow.sklearn.save_model(model, "model")

print("Linear Regression saved successfully!")
```

FASTAPI: (Model Deployment)

→In this project, **FastAPI** is used to deploy the trained machine learning model as a web API. FastAPI is a modern Python framework that allows developers to build high-performance APIs quickly and efficiently. It enables communication between the **frontend interface** and the **machine learning model**.

→The API receives input data from the user, processes it using the trained model, and returns the predicted output.

```
#Importing all required libraries

from fastapi import FastAPI

import mlflow.sklearn

import pandas as pd

import os

app = FastAPI()

# Set MLflow tracking to local folder

mlflow.set_tracking_uri("file:./mlruns")

# Load model using local path

model = mlflow.sklearn.load_model("model")

@app.get("/")

def home():

    return {"message": "Goodreads Prediction API Running"}

@app.post("/predict")
```

```
def predict(data: dict):
    df = pd.DataFrame([data])
    prediction = model.predict(df)
     return {"prediction": prediction.tolist()}
# CONNECTION TO THE FRONTEND
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Docker Containerization

→Docker is used to package the entire application into a container. The Docker image includes the application code, dependencies, and runtime environment.

→Containerization ensures that the application behaves the same regardless of where it is deployed. Whether it runs on a developer's laptop, a testing server, or a production environment, Docker ensures consistency.

→The Docker file defines the steps required to build the container image, including installing dependencies and starting the FastAPI server.

```
#DOCKER CONTAINER
  # Use official Python image
FROM python:3.11-slim
# Set working directory inside container
WORKDIR /app
```

```
# Copy requirements first (better Docker caching)

COPY requirements.txt .

# Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

# Copy all project files

COPY . .

# Expose FastAPI port

EXPOSE 8000

# Start FastAPI using uvicorn

CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Requirement.txt

→The **requirements.txt** file is used to list all the Python libraries required for the project. It helps ensure that the application runs with the correct dependencies when deployed on another system or inside a Docker container.

→ By installing the libraries listed in this file, developers can easily recreate the same development environment.

```
mlflow==3.10.0

cloudpickle==3.1.2

scikit-learn==1.8.0

fastapi

uvicorn

pandas

numpy
```

Environment Configuration

→The project includes configuration files that define the runtime environment.

→ Therequirements.txt contains the Python dependencies required for running the application.

→conda.yaml defines the Conda environment used during model training.

→python_env.yaml defines the Python version and dependency configuration required to run the model.

→These files help recreate the same environment when the project is executed on another machine.

python_env.yaml File

Code

```
python: 3.11.6
build_dependencies:
- pip==26.0.1
- setuptools==65.5.0
- wheel
dependencies:
- -r requirements.txt
python_env
```

Explanation of python_env.yaml

→The **python_env.yaml** file defines the Python environment configuration required to run the machine learning model. It ensures that the correct Python version and required dependencies are installed when the project is deployed.

→This file helps maintain a consistent environment so that the application behaves the same way on different systems.

Python Version

```
python: 3.11.6
```

→This line specifies the Python version used in the project. Using a fixed version ensures compatibility with the machine learning libraries and prevents errors during deployment.

Build Dependencies

```
build_dependencies:
- pip==26.0.1
- setuptools==65.5.0
- wheel
```

These are tools required for installing and managing Python packages.

- **pip** is used to install Python libraries.
- **setuptools** helps build and distribute Python packages.
- **wheel** is used to install packages more efficiently.

Project Dependencies

```
dependencies:
- -r requirements.txt
```

This line tells the environment to install all the libraries listed in the **requirements.txt** file.
The requirements file contains the main project dependencies such as **MLflow, FastAPI, Scikit-learn, Pandas, and NumPy**.

Role of python_env.yaml in the Project

→The **python_env.yaml file ensures that the machine learning application runs with the correct environment settings**. It helps maintain consistency between development and deployment environments and prevents compatibility issues.

→This file is particularly useful when deploying the application using **MLflow or Docker containers**, as it allows the exact Python environment to be recreated automatically

Testing and Validation

→Once the API is developed, it can be tested using several tools such as Swagger UI, Postman, or curl commands.

→Swagger UI is automatically generated by FastAPI and provides a simple interface for testing API endpoints.

→Testing ensures that the API correctly receives input data, processes it using the model, and returns the expected predictions.

FRONTEND:

→The frontend of the SmartReads AI system is developed using **HTML, CSS, and JavaScript**. It provides a user-friendly interface that allows users to enter book details and obtain predicted scores from the machine learning model. The frontend communicates with the backend API built using FastAPI to send user inputs and display prediction results.

The interface consists of three main components:

- **HTML** for structuring the web page
- **CSS** for styling the interface
- **JavaScript** for connecting the frontend with the backend API

HTML Structure

The HTML file defines the structure of the web page and creates the layout for the prediction interface.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SmartReads AI - Predict Score</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
```

The `<head>` section contains the metadata and links the CSS stylesheet used for styling the page.

Input Form for Book Details

The main part of the page contains an input form where the user enters book information.

```
<label>Ratings Count</label>
<input type="number" id="ratingsCount">

<label>Current Average Rating</label>
<input type="number" step="0.01" id="averageRating">
```

These input fields collect the **ratings count and average rating**, which are the main features used by the machine learning model to predict the book score.

Prediction Button

```
<button onclick="generatePrediction()">Generate Prediction</button>
```

This button triggers the JavaScript function **generatePrediction()**, which sends the input data to the backend API for prediction.

Displaying Prediction Results

```
<div class="prediction-value" id="prediction">--</div>
```

This section displays the predicted score returned by the machine learning model.

The result will update dynamically after the prediction is generated.

JavaScript for API Communication

The JavaScript file is responsible for connecting the frontend interface to the backend prediction API.

```
async function generatePrediction() {
```

This function runs when the user clicks the **Generate Prediction** button.

Reading User Input

```
const ratingsCount =
parseFloat(document.getElementById("ratingsCount").value);
const averageRating =
parseFloat(document.getElementById("averageRating").value);
```

This code retrieves the values entered by the user and converts them into numeric format so they can be used by the machine learning model.

Sending Request to Backend

```
const response = await fetch("http://127.0.0.1:8000/predict", {
method: "POST",
headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({
RatingsCount: ratingsCount,
AverageRating: averageRating
})
});
```

The **fetch API** is used to send a POST request to the FastAPI backend.

The input values are converted into **JSON format** and sent to the `/predict` endpoint.

Displaying Prediction Result

```
const data = await response.json();
document.getElementById("prediction").innerText =
data.prediction[0].toFixed(2);
```

After receiving the response from the backend, the predicted score is displayed on the webpage.

The result is formatted to two decimal places for better readability.

CSS Styling

The CSS file is used to design the layout and improve the visual appearance of the application.

```
body {
margin: 0;
font-family: Arial, sans-serif;
background: #f4f4f9;
}
```

This sets the background color and default font style of the page.

Sidebar Styling

```
.sidebar {
width: 250px;
background: #5b2be0;
color: white;
padding: 20px;
height: 100vh;
}
```

The sidebar provides a navigation panel and gives the application a structured layout.

Card Layout for Input and Results

```
.card {
background: white;
padding: 25px;
border-radius: 12px;
box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}
```

The **card design** organizes the input form and prediction results into visually separate sections.

Prediction Result Highlight

```
.prediction-value {
font-size: 50px;
font-weight: bold;
color: #6c3df4;
}
```

This style highlights the predicted score using a larger font size and color to make it clearly visible to the user.

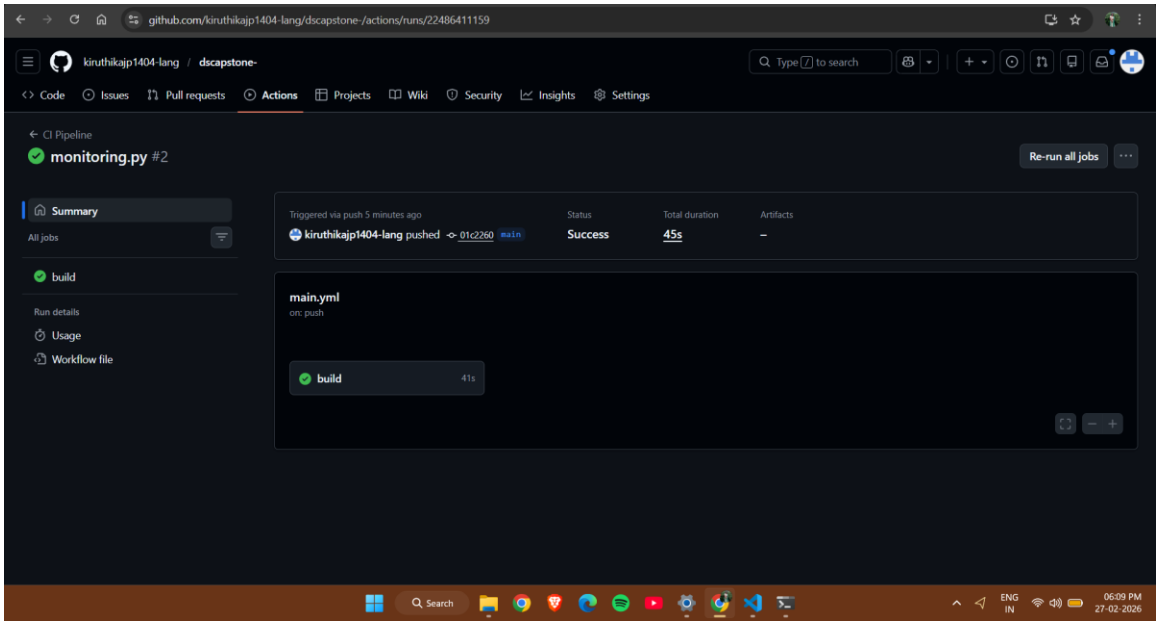
Overall Workflow

The frontend works together with the backend system using the following workflow:

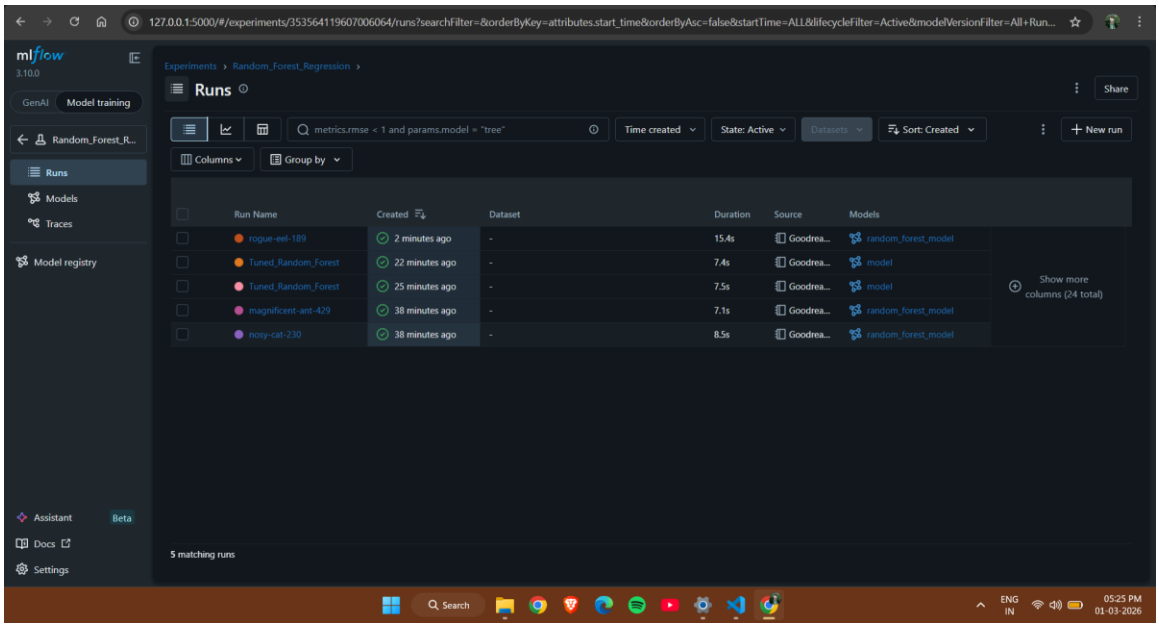
User Input → JavaScript Function → FastAPI Backend API → Machine Learning Model → Prediction Result → Display on Web Interface

This integration enables users to easily interact with the machine learning model through a web-based interface and obtain real-time predictions.Bottom of Form.

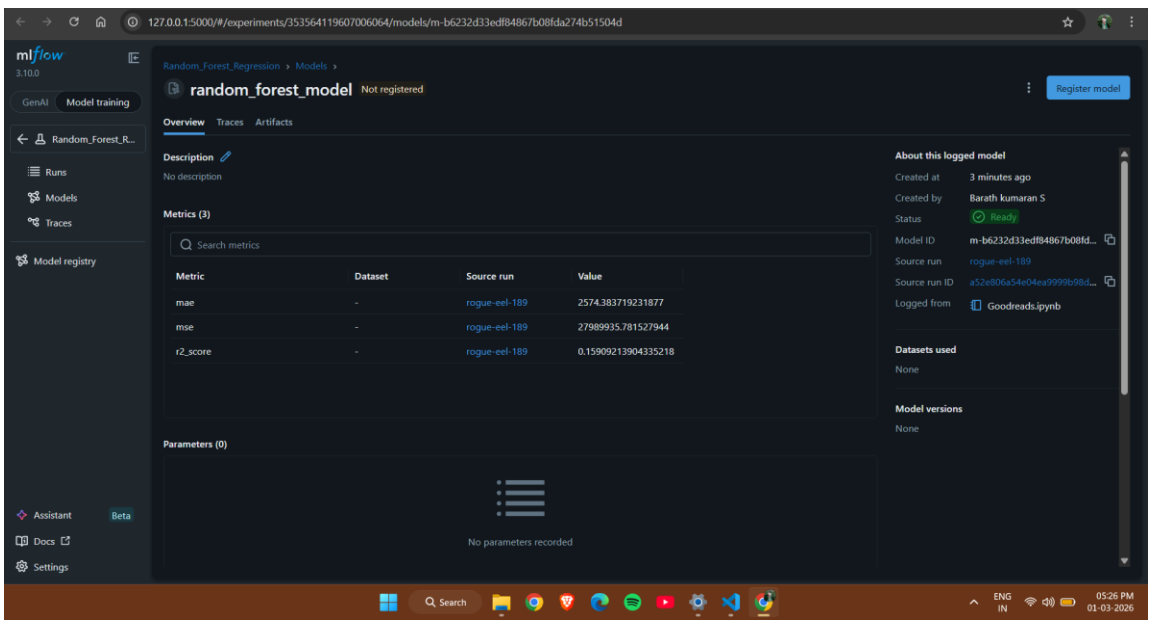
SCREENSHOTS:



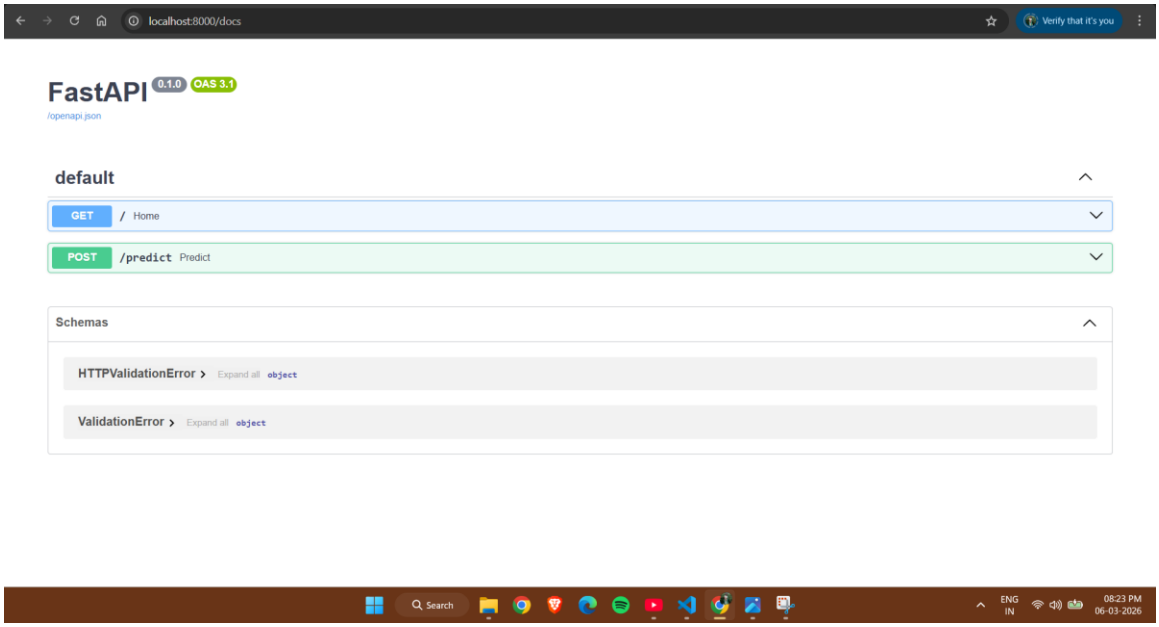
Monitoring



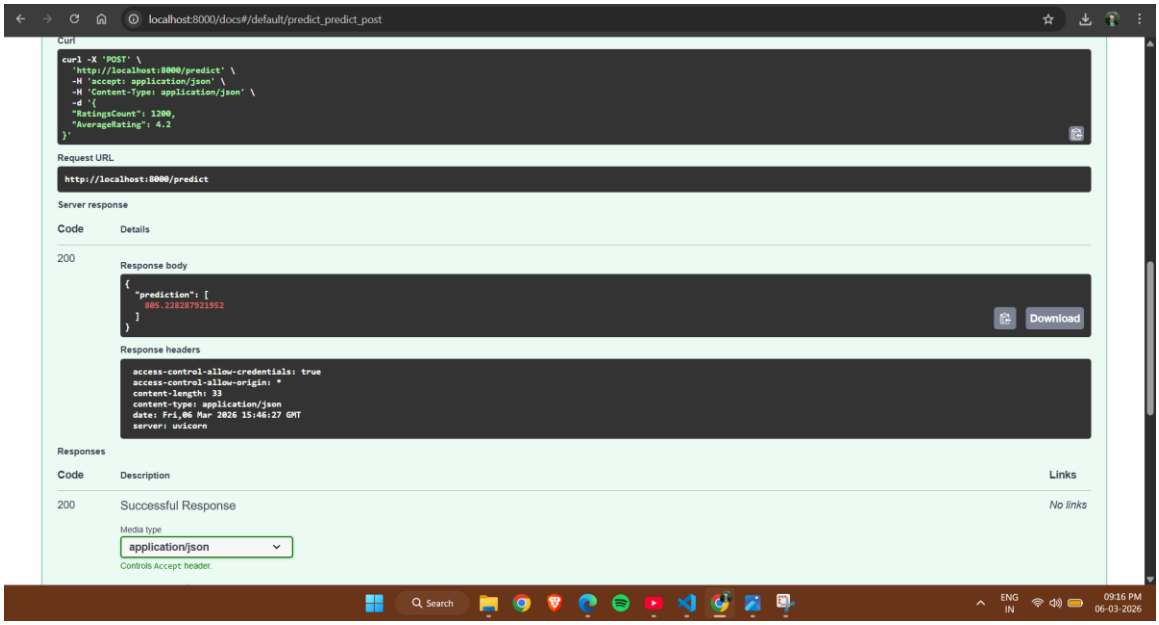
MLRUNS



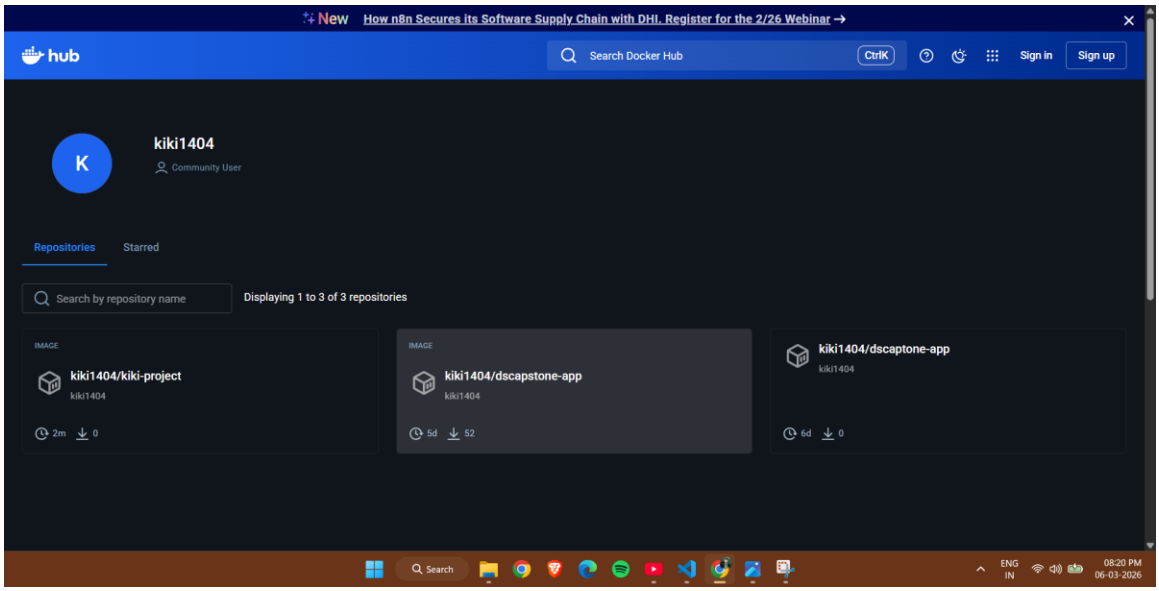
Runned models values



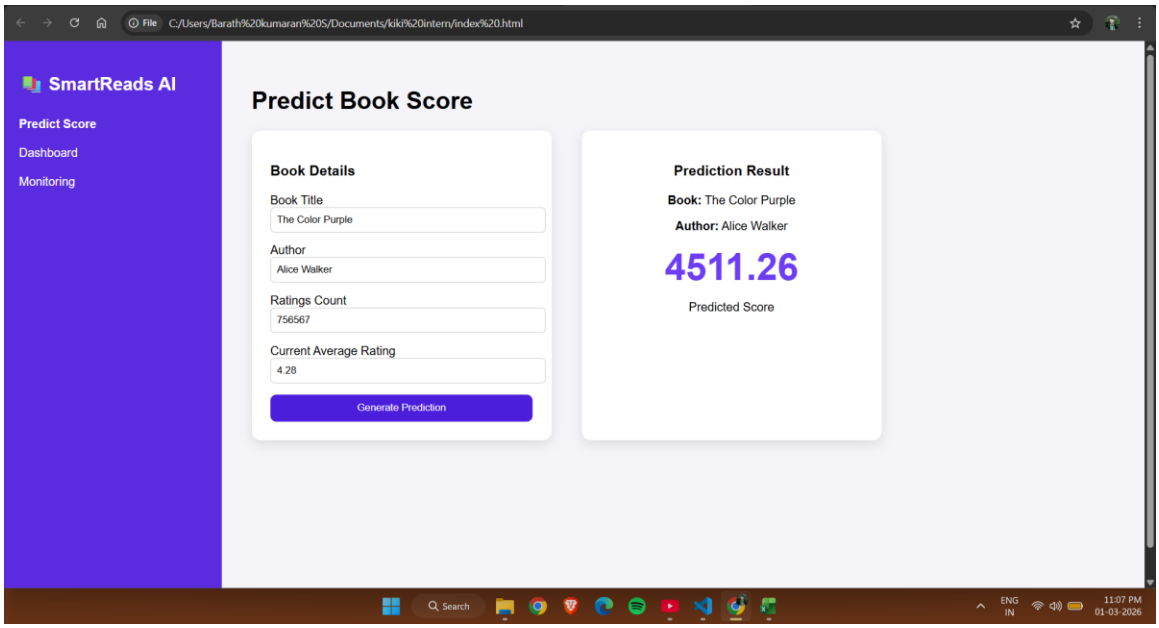
FAST API



VALUE PREDICTION PAGE



DOCKER IMAGE



PREDICTION PAGE

Future Improvements

1. There are several ways this project can be improved in the future.

2. More features from the dataset can be used to improve prediction accuracy. Advanced machine learning models or deep learning approaches can also be explored.

3.A user interface can be developed so that users can interact with the system through a graphical interface.

4.The system can also be deployed to cloud platforms such as AWS, Azure, or Google Cloud to make it accessible over the internet.

CONCLUSION:

- The **Book score prediction project** successfully demonstrates the use of machine learning techniques to predict book scores based on rating count and average rating.
- The project focuses on analyzing book rating data and building a predictive model that can estimate the likely score of a book using historical data.
- A machine learning model was trained using the dataset to understand the relationship between user ratings and overall book performance.
- The trained model was integrated with a **FastAPI backend**, which allows the prediction system to be accessed through an API endpoint.
- A simple and interactive **frontend interface** was developed using HTML, CSS, and JavaScript to allow users to input book data and view predictions.
- The frontend communicates with the backend API to send input values and receive prediction results in real time.
- The system demonstrates how machine learning models can be deployed and used in real-world applications through web technologies.
- This project highlights the importance of **data-driven decision making** in digital platforms such as online bookstores and recommendation systems.
- The book score prediction system provides a simple method for estimating book scores and can help users identify potentially popular books.
- In the future, the system can be improved by including additional features such as **author popularity, book genre, publication year, and user reviews**.
- More advanced machine learning models and larger datasets can further improve prediction accuracy.
- The system can also be deployed on **cloud platforms** to make it accessible to a larger number of users.
- Overall project successfully integrates **machine learning, backend API development, and frontend web technologies** to create a functional prediction system.